arXiv:2501.06220v1 [cs.LG] 7 Jan 2025

Powerful Design of Small Vision Transformer on CIFAR10

Gent Wu
Surrey Institute for People-Centred AI (PAI)
Guildford GU2 7XH, Surrey, United Kingdom
clouderow@gmail.com

Abstract

Vision Transformers (ViTs) have demonstrated remarkable success on large-scale datasets, but their performance on smaller datasets often falls short of convolutional neural networks (CNNs). This paper explores the design and optimization of Tiny ViTs for small datasets, using CIFAR-10 as a benchmark. We systematically evaluate the impact of data augmentation, patch token initialization, low-rank compression, and multi-class token strategies on model performance. Our experiments reveal that low-rank compression of queries in Multi-Head Latent Attention (MLA) incurs minimal performance loss, indicating redundancy in ViTs. Additionally, introducing multiple CLS tokens improves global representation capacity, boosting accuracy. These findings provide a comprehensive framework for optimizing Tiny ViTs, offering practical insights for efficient and effective designs. Code is available at https://github.com/erow/PoorViTs

Vision Transformers (ViTs) have achieved state-of-the-art results on several benchmarks. However, their performance gains heavily depend on large-scale training, either through supervised learning [6] or self-supervised learning [1]. The high computational cost associated with ViTs has hindered architectural innovation. Moreover, smaller ViTs often exhibit inferior effectiveness and efficiency compared to their convolutional counterparts. For instance, a Tiny ViT achieves only 80% accuracy on CIFAR-10, as reported by Cydia2018. While some studies have attempted to bridge the performance gap for small ViTs on small datasets, their solutions often fail to fully leverage the power of Transformers [22] by relying on convolutional techniques [18, 27]. For example, Zhang et al. improved ViT performance by incorporating depth-wise convolutions, achieving 96.41% accuracy. Other works have explored self-supervised learning to enhance feature representation [7, 15]. Our goal is to establish an efficient yet effective framework for training small ViTs on small datasets, facilitating research, particularly for beginners.

The CIFAR-10 dataset, introduced by Krizhevsky et al., has become a widely adopted benchmark for evaluating image classification models in machine learning. It comprises 60,000 32x32 color images distributed across 10 classes, with 6,000 images per class. The dataset is divided into 50,000 training images and 10,000 test images, providing a balanced platform for rapid experimentation and model validation. Despite its simplicity, CIFAR-10 remains a critical tool for the rapid validation of new algorithms. Recent advancements in training methods have significantly improved the speed and accuracy

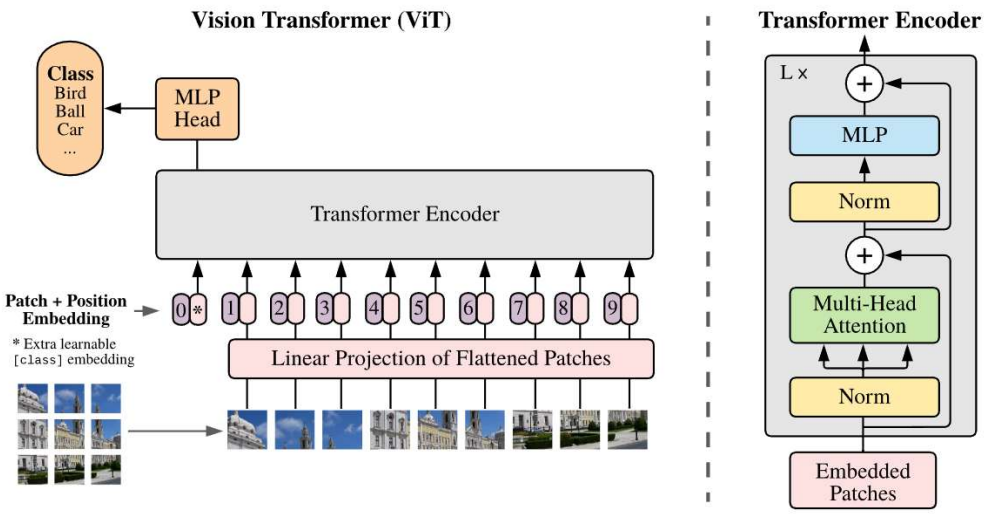


Figure 1. Vision transformer overview from [6]. It involves (1) **Tokenization**: The image is divided into patches of a predetermined size. Each patch is then linearly embedded, and position embeddings are incorporated to maintain spatial information. The sequence of vectors, which represents the embedded patches along with their positional data. (2) **Token transformation**: tokens are subsequently input into a conventional Transformer encoder. (3) **Task projection**: For the purpose of classification, a common method is employed, which involves appending an additional learnable "classification token" to the sequence of vectors. This token is trained to aggregate the information from all patches and serves as the basis for the classification decision.

of neural networks on CIFAR-10. For instance, Jordan introduced training methods that achieve 94% accuracy in just 3.29 seconds on a single NVIDIA A100 GPU. However, recent progress in efficient training on CIFAR-10 has primarily focused on convolutional neural networks (CNNs).

In this report, we investigate the architectural design of a Tiny ViT [7] to achieve competitive performance compared to CNN counterparts (93.58% on CIFAR10). Our findings reveal two key insights: 1) applying low-rank compression to queries does not degrade performance, and 2) reducing the dimensionality of patch tokens while maintaining the dimensionality of the CLS token does not diminish performance. These results suggest the presence of redundant information in ViTs, which can be exploited to improve efficiency without sacrificing accuracy.

1. Vision Transformer

Vision Transformer (ViT), as illustrated in Fig. 1, was first proposed by Dosovitskiy, drawing inspiration from the success of Transformers [22] in natural language processing. Transformers utilize an attention mechanism to model the relationships between tokens effectively. In ViTs, images are split into small patches and processed to extract high-level features. Specifically, a ViT has the following key procedures: image tokenization, token transformation, and task projection.

1.1. Patch Token = Patch Embedding + Positional Embedding

Tokenization generates patch tokens through patch embedding and positional embedding. Specifically, images are split into several patches, and each patch token represents the semantic information at a specific position. Initially, patch tokens only contain low-level pixel information. To differentiate patches with similar colors or textures, positional embeddings are essential. For instance, without positional

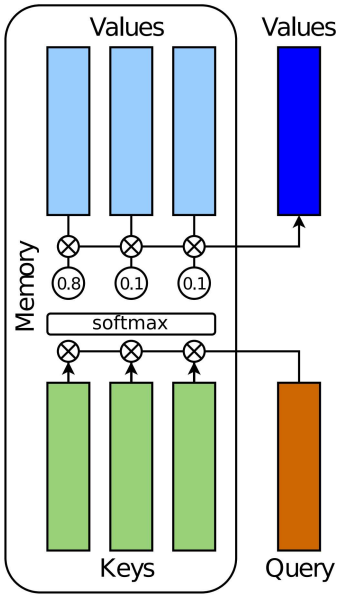


Figure 2. Overview of Attention mechanism.

embeddings, two patches from a white T-shirt and a white bag would be indistinguishable and fail to represent different objects (T-shirt and bag here). The tokenization can be formalized as: input image $\mathbf{x}_0 \in \mathbb{R}^{3\times H\times W}$ is rearranged into a patch sequence $\mathbf{x} \in \mathbb{R}^{L\times(3\times P^2)}$, where P is the patch size and $L = HW/P^2$ is the number of patches. A patch embedding layer projects each patch into a higher-dimensional space to obtain the patch embeddings $\mathbf{x}_e \in \mathbb{R}^{L\times C}$, where C is the embedding dimension. Position embeddings $\mathbf{x}_p \in \mathbb{R}^{L\times C}$ are then added to the patch embeddings to incorporate spatial information, resulting in the final patch tokens $\mathbf{x}_{\text{token}} = \mathbf{x}_e + \mathbf{x}_p$.

1.2. Transformer Block = Attention + Feed-forward Network

Transformer Block aggregates information carried by tokens globally to refine the tokens into higher semantic representations. Tokens will better represent the concepts for the patches as they pass through more transformer blocks. A transformation block consists of an Attention layer and a Feed-forward Network (FFN) with residual connections.

1.2.1. Multi-head Attention

The Attention mechanism in Vision Transformers allows the model to weigh the importance of different patches relative to each other. This is achieved through multi-head attention, where the model can attend to different positions in the image simultaneously. The mechanism involves generating query ($\mathbf{Q}$), key ($\mathbf{K}$), and value ($\mathbf{V}$) vectors from the input tokens. Fig. 2 illustrates the basic attention mechanism. The attention weights are computed using the scaled dot-product attention formula:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}, \tag{1}$$

where d_k is the dimensionality of the key vectors. The softmax function ensures that the attention weights sum to 1, enabling the model to focus on relevant patches for each token. Multi-head attention extends this by computing attention in parallel over multiple heads, allowing the model to capture diverse

relationships:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O, \tag{2}$$

where each head is computed as $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$, and $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, $\mathbf{W}_i^V$, and $\mathbf{W}^O$ are learnable projection matrices. $\mathbf{W}^O$ is able to fuse the learned information from these heads.

1.2.2. Feed-forward Network

The Feed-forward Network (FFN) is a standard MLP applied independently to each patch token. It helps in modeling complex relationships and transformations of the features extracted by the Attention layer. The FFN typically consists of two linear layers with an activation function, ReLU or GELU. The FFN can be expressed as:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2^T \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) + \mathbf{b}_2, \tag{3}$$

where $\mathbf{W}_1$, $\mathbf{W}_2$ are learnable weight matrices, and $\mathbf{b}_1$, $\mathbf{b}_2$ are bias terms, $\sigma(\cdot)$ denotes an activation function, e.g., ReLU. Like the Attention layer, it uses residual connections and layer normalization to facilitate better training and convergence.

The FFN can be treated as a memory-based attention module [8] in which the first layer is a set of keys and the second layer stores the corresponding keys.

1.3. Task-related Head

The Projection Head is a critical component in Vision Transformers (ViTs) for adapting the model to different downstream tasks, such as classification [21] and dense prediction [17]. In this report, we adopt classification as an example to demonstrate how to utilize the extracted features from tokens for downstream tasks.

A [CLS] token is inserted alongside the patch tokens before being processed by the Transformer blocks. The [CLS] token serves as a global representation of the entire image, enabling the model to focus on global information rather than local information tied to specific positions. After passing through the Transformer blocks, the final representation of the [CLS] token (or a global pooling of all tokens) is fed into a projection head. This projection head maps the high-dimensional features to a task-specific output space.

The projection head is typically implemented as a simple multi-layer perceptron (MLP). For a classification task with C classes, the projection head can be formulated as:

$$\mathbf{p} = \text{MLP}(\mathbf{h}_{\text{[CLS]}}) = \mathbf{W}_2^T \sigma(\mathbf{W}_1 \mathbf{h}_{\text{[CLS]}} + \mathbf{b}_1) + \mathbf{b}_2, \tag{4}$$

where $\mathbf{h}_{\text{[CLS]}} \in \mathbb{R}^C$ is the final representation of the [CLS] token, $\mathbf{W}_1 \in \mathbb{R}^{D \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times D}$ are learnable weight matrices, $\mathbf{b}_1 \in \mathbb{R}^D$ and $\mathbf{b}_2 \in \mathbb{R}^C$ are bias terms, and ReLU is the activation function. The output $\mathbf{p} \in \mathbb{R}^C$ represents the logits for each class.

The model is trained using the cross-entropy loss, which measures the discrepancy between the predicted class probabilities and the ground truth labels. The cross-entropy loss is defined as:

$$\mathcal{L} = -\sum_{i=1}^{C} \mathbf{p}_i \log(\text{softmax}(\mathbf{y})_i), \tag{5}$$

where $\mathbf{y} \in \{0, 1\}^C$ is the one-hot encoded ground truth label, and $\text{softmax}(\mathbf{p})_i$ is the predicted probability for class i.

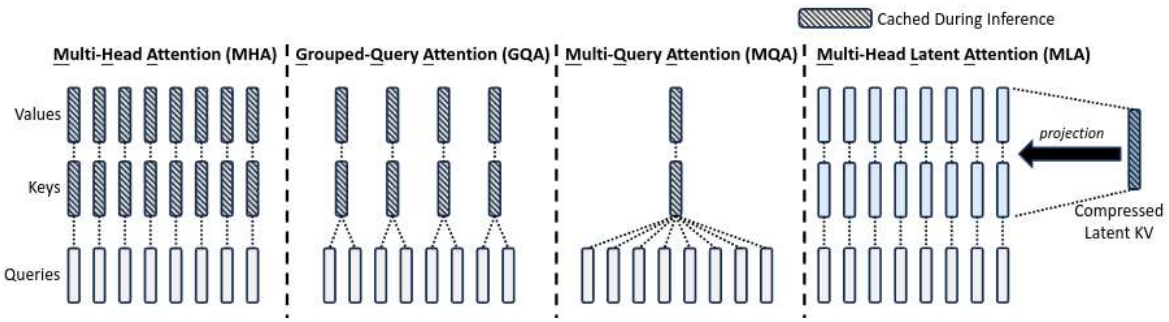


Figure 3. Illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA) from [14].

1.4. Modern Techniques

The computational cost of the standard attention mechanism has a complexity of $O(L^2d+nd^2)$, where L is the number of tokens. This quadratic complexity can become a bottleneck, especially when processing high-resolution images or long sequences. To address this, many modern techniques have been proposed to improve the efficiency of attention mechanisms. These techniques include sparse attention [3], linear attention [12], and low-rank compression [14]. In this report, we focus on a mechanism called **multi-latent attention**(MLA), which compresses matrices to reduce KV cache while maintaining the ability to capture global dependencies.

Multi-latent Attention. The core of MLA is the low-rank compression to reduce active memory by a down-projection matrix W^D and an up-projection matrix W^U. For instance, we can perform low-rank compression for the queries:

$$\begin{aligned} c &= W^D x, \\ q &= W^U c, \end{aligned} \tag{6}$$

where $c \in \mathbb{R}^{d_c}$ is the compressed latent vector; d_c denotes the compression dimension, which is far smaller than the original dimension.

2. Experiment

2.1. Tiny Vision Transformer

Baseline. The baseline code is based on [7]. A variety of data augmentation methods are applied, including CutMix [25], MixUp [26], AutoAugment (aa) [9], and Repeated Augment [4, 9]. Additionally, regularization methods such as label smoothing [19], stochastic depth [10], and random erasing [28] are employed. All models are trained for 100 epochs with a batch size of 256 on a single NVIDIA 3090 GPU paired with an AMD EPYC 7532 32-Core Processor. The Adam optimizer is used with a cosine learning rate schedule, starting at 0.002 and decayed using a cosine strategy with a 10-epoch warmup. The weight decay rate is set to 5e-2. The Tiny Vision Transformer consists of 12 attention heads, 9 transformer blocks, and 64 patches for images (each patch size is 4x4). The experimental results are summarized in Tab. 2.

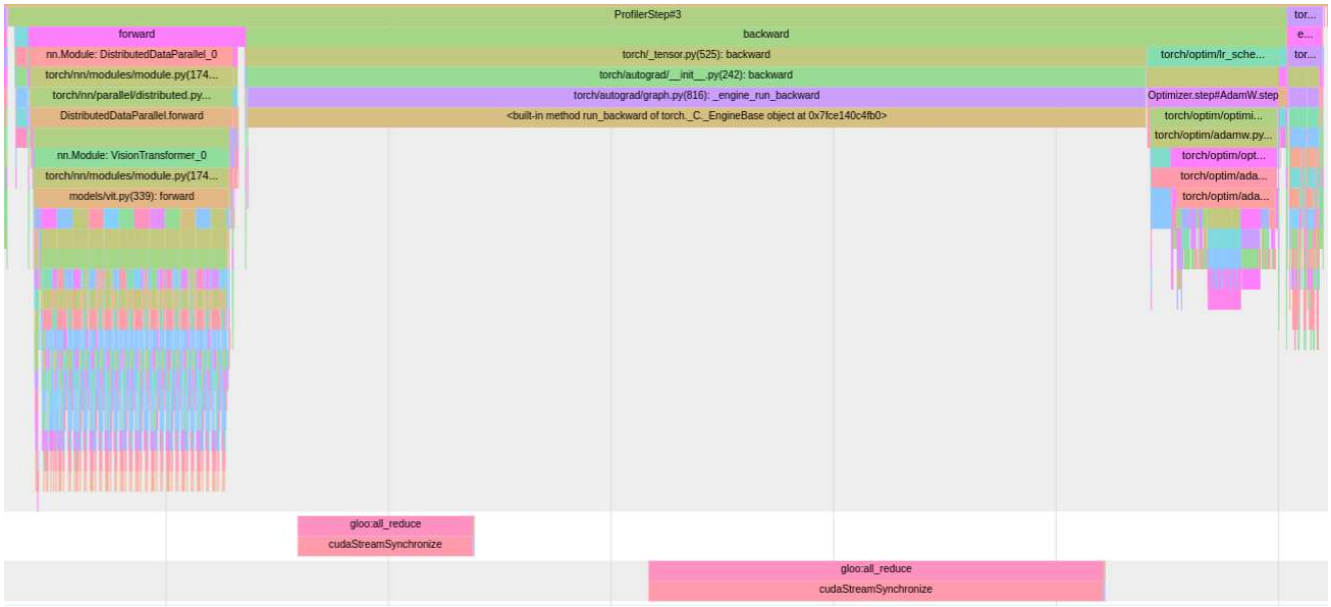


Figure 4. Tracing one training step. One step (120.7 ms) involves forward (25.2 ms), backward (79.3 ms), optimization (12.7 ms), and other processes.

Profiler. To identify the training bottleneck, we employ the PyTorch Profiler [20]. Fig. 4 visualizes the time span of each procedure using tracing[1]. The backward process consumes the majority of the training time, indicating that the bottleneck lies in the high computational cost of handling large feature maps during both forward and backward passes. Tab. 1 compares the forward time across different batch sizes.

Batch Size (bs)	**32**	**64**	**128**	**256**	**512**	**1024**
Throughput (images/s)	3162.09	6521.61	9449.16	9721.13	10368.24	10607.06
Memory Usage (GB)	0.31	0.62	1.25	2.50	5.00	9.98

Table 1. Benchmark of Tiny ViT on an NVIDIA RTX 3090 GPU.

	Baseline	**Augmentation**			**Patch**		**MLA**					**DDP**		
		-aa	-mixup	-cutmix	sin	Witen	qkv	kv	qk	q	k	bs256	bs1024	
Runtime	4658	4684	4656	4651	4687	4676	4805	4756	4785	4716	4711	3582	1371	1545
Val/Acc	93.65	92.14	93.68	92.77	93.43	93.28	91.8	92.43	92.42	93.32	92.5	93.44	92.09	92.38

Table 2. Summary of our experiments. 'aa' denotes AutoAugment. Data augmentation is critical except for Mixup. Learnable positional embeddings demonstrate the best performance. Compressing queries during training exhibits the least performance drop. Increasing batch size reduces training time when Distributed Data Parallel (DDP) is applied, but at the cost of diminished performance. Lion [2] is more effective than AdamW [16] for optimizing the Tiny ViT.

[1] `edge://tracing/` in the Edge browser.

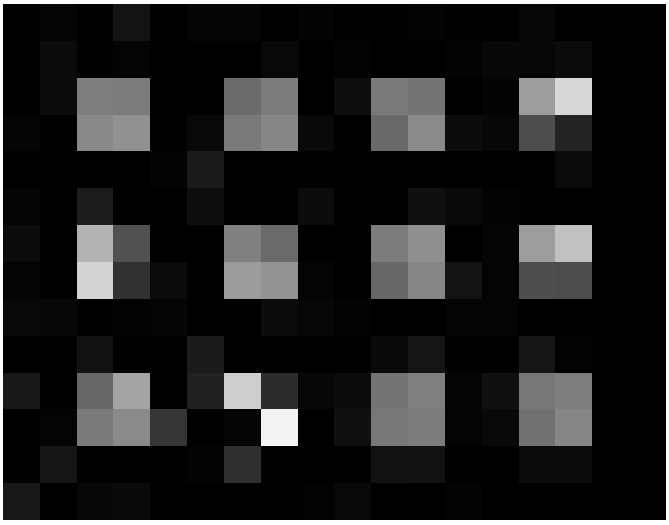


Figure 5. Whitening patterns in patch embedding.

2.2. Ablation Study

Augmentation. The experiments demonstrate the significance of data augmentation methods in improving validation accuracy. Removing AutoAugment ('-aa') results in a noticeable drop in accuracy (92.14 vs. 93.65), highlighting its importance. Similarly, disabling CutMix ('-cutmix') leads to a decline (92.77 vs. 93.65). Interestingly, removing MixUp ('-mixup') marginally improves accuracy (93.68), suggesting it may not be as beneficial in this setup. These results underline the importance of selecting appropriate augmentations tailored to the dataset and architecture.

Patch Token Initialization. We utilize learnable positional embeddings by default. The positional embeddings can be replaced by sinusoidal positional embeddings ('sin'), which are non-learnable and fixed during training. Sinusoidal embeddings achieve slightly worse validation accuracy (93.43 vs. 93.65). We follow ysam code to initialize the patch embedding using whitening initialization ('Witen') instead of random initialization. Whitening initialization identifies patterns in the dataset, as illustrated in Fig. 5, to speed up the training process. However, this technique, while effective in CNNs, does not translate well to ViTs.

Distributed Data Parallel (DDP). We conduct distributed data parallel training with 4 NVIDIA 3090 GPUs. Experiments with DDP show that increasing the batch size significantly reduces training time (e.g., from 3582s to 1371s with 'bs1024'). However, larger batch sizes lead to diminished validation accuracy (92.09 vs. 93.65 for 'bs256'). This trade-off emphasizes the need to balance computational efficiency with model performance when scaling training.

Optimizer. The Lion optimizer [2] is compared against AdamW [16] for optimizing the Tiny ViT. Fig. 6 compares Lion to Adam in terms of finding a good solution. Lion achieves higher accuracy (92.38), demonstrating its superior capability for this architecture.

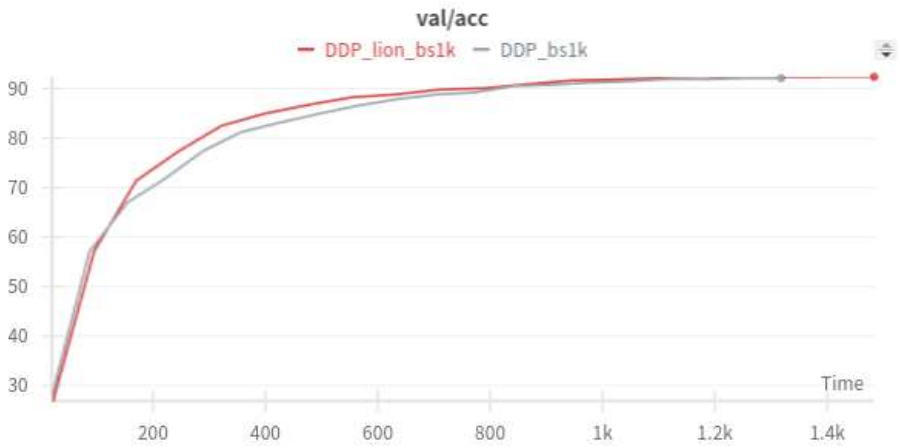


Figure 6. Accuracy over training time with the Lion optimizer. Although Lion consumes more time per step, it requires fewer iterations to find a good solution.

dim	192	96	96
num CLS	1	1	2
acc	93.65	90.41	93.95

Table 3. Multi-class token experiments.

2.3. Low-dimensional Patch Tokens

Low-Rank Compression. Different configurations of Multi-Head Latent Attention (MLA) [14] were evaluated, including modifications to the queries ('q'), keys ('k'), or values ('v'). Compressing all three ('qkv') incurs a noticeable performance drop (91.8 vs. 93.65), while modifications targeting specific combinations (e.g., 'kv', 'qk', 'q', and 'k') yield better results. In particular, compressing 'q' results in only a marginal drop in precision, consistent with the findings in [14]. These results suggest the potential for low-rank compression in ViTs to reduce activation memory.

Multi-Class Token. We investigate potential solutions to improve the efficiency of ViTs. One natural approach is to reduce the dimensionality of tokens. However, lower-dimensional tokens imply reduced capacity for visual information, as observed in Tab. 3, where reducing the dimension to 96 results in a 3.24% drop in accuracy. We argue that patch tokens, which are numerous (64 vs. 1), have a higher capacity than the CLS token. Even though patch tokens can represent the image effectively, these semantic representations are challenging to express through a narrow CLS token. To address this, we introduce additional CLS tokens to capture global representations. These CLS tokens are concatenated and fed to the classifier. Our simple multi-class token (MCLS) strategy effectively boosts the capacity of global representation. Tab. 3 validates our hypothesis, showing that using 2 CLS tokens improves accuracy from 90.41 to 93.95. Note that our MCLS approach differs significantly from Mctformer+ [23], where class information is distributed across multiple CLS tokens. In contrast, our proposal simply expands the dimensionality of the CLS token, with each CLS token representing a portion of the original CLS token.

References

[1] Sara Atito, Muhammad Awais, and Josef Kittler. Sit: Self-supervised vision transformer. *arXiv preprint arXiv:2104.03602*, 2021. 1

[2] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, 2023. 6, 7

[3] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019. 5

[4] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation strategies from data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 113–123, 2019. 5

[5] Cydia2018, 2021. https://github.com/Cydia2018/ViT-cifar10-pruning. 1

[6] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1, 2

[7] Hanan Gani, Muzammal Naseer, and Mohammad Yaqub. How to train vision transformer on small-scale datasets? *arXiv preprint arXiv:2210.07240*, 2022. 1, 2, 5

[8] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2020. 4

[9] Elad Hoffer, Tal Ben-Nun, Itay Hubara, Niv Giladi, Torsten Hoefler, and Daniel Soudry. Augment your batch: Improving generalization through instance repetition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8129–8138, 2020. 5

[10] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 646–661. Springer, 2016. 5

[11] Keller Jordan. 94% on cifar-10 in 3.29 seconds on a single gpu. *arXiv preprint arXiv:2404.00498*, 2024. 2

[12] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020. 5

[13] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 1

[14] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024. 5, 8

[15] Yahui Liu, Enver Sangineto, Wei Bi, Nicu Sebe, Bruno Lepri, and Marco Nadai. Efficient training of visual transformers with small datasets. *Advances in Neural Information Processing Systems*, 34:23818–23830, 2021. 1

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. 6, 7

[17] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12179–12188, 2021. 4

[18] Ran Shao and Xiao-Jun Bi. Transformers meet small datasets. *IEEE Access*, 10:118454–118464, 2022. 1

[19] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 5

[20] Torch. PyTorch Profiler. `https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html`. [Accessed 06-01-2025]. 6

[21] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 4

[22] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. 1, 2

[23] Lian Xu, Mohammed Bennamoun, Farid Boussaid, Hamid Laga, Wanli Ouyang, and Dan Xu. Mctformer+: Multi-class token transformer for weakly supervised semantic segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 2024. 8

[24] ysam code, 2023. https://github.com/tysam-code/hlb-CIFAR10. 7

[25] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6023–6032, 2019. 5

[26] Hongyi Zhang. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412*, 2017. 5

[27] Tianxiao Zhang, Wenju Xu, Bo Luo, and Guanghui Wang. Depth-wise convolutions in vision transformers for efficient training on small datasets. *Neurocomputing*, 617:128998, 2025. 1

[28] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI conference on artificial intelligence*, pages 13001–13008, 2020. 5